



Università di Pisa

Dipartimento di Informatica

Shakespeare Interpreter in Go

AI-Assisted Development

A Documented Ledger of the Prompts Used to Drive an LLM
Through Every Phase of Compiler Construction

Author

Lorenzo Bandini

Course

Advanced Programming
A.Y. 2025/2026

Professors

Andrea Corradini
Antonio Cisternino

July 22, 2026

Contents

1	Approach and Methodology	2
1.1	The Prompt Engineering Strategy	2
1.1.1	How Planning Worked in Practice	2
1.2	The 6-Section Plan Mode Scaffold	2
1.3	Cross-Model Triangulation and Decision Discipline	3
1.4	Canonical-Source Adjudication	3
1.5	The Tool Harness	3
1.6	The Iterative Generate-Verify-Accept Loop	4
2	Phase 0 - Ideation, Language Choice, and the Go Pivot	4
2.1	The Language Question	4
2.2	The Go Pivot	4
2.3	Scaffolding and Design	5
2.4	Prerequisite Gap Analysis	5
3	Phase 1 and 2 - Lexer and Parser	6
3.1	The Plan-Mode Prompt	6
3.2	Four Clarifying Questions	7
3.3	Implementation	8
4	Phase 3 - Semantic Analysis	8
4.1	Prerequisite Bugfix Interjection	8
4.2	The Plan-Mode Prompt	8
4.3	Critical Decisions	10
4.4	Implementation	11
5	Phase 4 - Runtime	11
5.1	The Plan-Mode Prompt	11
5.2	Implementation	13
6	Phase 5 - CLI Integration	13
6.1	The REPL Interaction Model Question	13
6.2	Six Subcommands	13
6.3	Implementation	14
7	Phase 6 - Documentation	14
8	Phase 7 - The REPL-Resolution Crisis	14
8.1	The Fixes	14
8.2	Implementation	15
9	Phase 8 - Final Comprehensive Audit	15
9.1	Implementation	15
10	Cost Ledger Summary	15

This document catalogues the prompt stream used to drive an LLM through every phase of building a Shakespeare Programming Language (SPL) interpreter in Go. It is structured as a first-person reflective narrative of the AI collaboration journey: the decisions, the thinking, the crises, and the recoveries.

A companion Validation Document discusses how the generated output was verified; this document focuses on *how* the prompts were crafted, *why* architectural forks were resolved one way rather than another, and *what* each phase cost in token-economics terms.

1 Approach and Methodology

The complete project lives at <https://github.com/lorenzobandini/shakespeare-interpreter-go> and the documentation site is at <https://lorenzobandini.github.io/shakespeare-interpreter-go/>.

Before describing what was built phase by phase, it is necessary to explain *how* the project was approached. The methodology that emerged through the first few phases turned out to be as important as the code itself.

1.1 The Prompt Engineering Strategy

The core strategic insight was a **planner/executor cost optimization**, validated over the entire project lifespan:

- **Planner model:** A frontier open model (GLM 5.2) wrote detailed architectural plans. It asked clarifying questions, forced numbered-option decisions, and produced comprehensive step-by-step execution documents.
- **Executor model:** A cheaper model (DeepSeek V4 Flash, a fraction of the cost) followed the plans and wrote code. It never made architectural decisions, it only executed.

This separation reduced implementation cost dramatically. The planning model spent tokens thinking deeply about structure; the implementation model spent tokens only on code, not on re-deriving architectural reasoning.

1.1.1 How Planning Worked in Practice

For example, the initial planning prompt for the lexer and parser began by establishing a strict “PLAN MODE” role:

You are an expert Principal Software Engineer and System Architect specializing in Compiler Design and the Go programming language. You operate strictly in PLAN MODE. Your job is to create an incredibly detailed, foolproof, step-by-step execution plan for implementing a Shakespeare Programming Language (SPL) interpreter in Go.

It then defined the required output format and a strong negative constraint: “Do **NOT** write full function implementations... you may define Go struct definitions, interface signatures, or bulleted algorithmic logic, but you must never provide complete code blocks meant for copy-pasting.” This separation kept architecture and implementation genuinely decoupled.

1.2 The 6-Section Plan Mode Scaffold

Every phase prompt reused the same six-section structure, refined across iterations:

1. **Role and System Context** - Expert role declaration with a “strictly in PLAN MODE” mandate.
2. **Project Status and Architecture** - What is already completed, what this phase builds on.
3. **Available Context Files** - Exactly which files the model must read and update (`progress.md`, `error_taxonomy.md`, `spl_specification.md`).
4. **Shared Infrastructure** - Logger configuration, testing strategy.
5. **Instructions and Plan Requirements** - Micro-step unrolling, TDD requirement, strict negative constraint.
6. **Expected Output Format** - Hierarchical markdown checklist structure.

1.3 Cross-Model Triangulation and Decision Discipline

This was the central human-in-the-loop mechanism that kept decisions deliberate and well-reasoned:

1. GLM 5.2 asks numbered clarifying questions with options (1/2/3 plus “Type your own answer”).
2. The questions are copied into a Claude Sonnet 5 side-chat holding full project context (`PROGRESS.md`, `specification.md`, `errors.md`).
3. Sonnet produces an independent curated analysis with concrete answers, highlighting risks and trade-offs.
4. The curated answer is pasted back into opencode for GLM.

Using a second model with complete context acted as a circuit breaker on the planning model. It caught several critical errors before any code was written, and more importantly, it ensured that every architectural fork was evaluated against the real spec, not the model’s memory.

1.4 Canonical-Source Adjudication

The reference specification `docs/spl/specification.md` was the absolute source of truth. It had been built from two canonical references, the Grokipedia/Esolang community, maintained SPL reference and the 2001 Åslund–Hasselström SPL manual, and the LLM was never allowed to override it with auto-generated paraphrase. Whenever the model’s auto-generated suggestions conflicted with the spec, the spec won. Other guiding files are `error_taxonomy.md` for error formatting and `progress.md` for state tracking that were also referenced in every plan cycle, creating a stable foundation that prevented drift.

1.5 The Tool Harness

The project used **opencode** (an AI-powered coding harness that orchestrates multiple models and skills; see <https://opencode.ai>) throughout planning and development. A paid Go plan gave access to frontier models and unlimited usage of the most demanding skills. Key capabilities:

- **Superpowers skill** - Forced structured planning before every implementation phase, mandated clarifying questions, prevented premature coding.
- **Headroom (MCP)** - Deterministic log compression and retrieval for shrinking verbose build/test logs without losing signal.

- **Ponytail skill** - Token-efficiency mode forcing minimal implementations, YAGNI decisions, deletion over addition.
- **ctx7 / context7** - Fetched up-to-date library documentation (Cobra, Task, Go stdlib patterns) so the model never relied on stale training data.
- **Custom skills:** `golang-patterns` (idiomatic Go), `golang-testing` (table-driven tests, golden files), `security-review` (auth/secrets/input checklist).
- **Language servers:** Go (`gopls`) and YAML LSP for real-time in-editor feedback.

1.6 The Iterative Generate-Verify-Accept Loop

Every phase followed a tight workflow that blended the planner/executor split with immediate validation:

1. **Generate (Plan mode):** Scaffold prompt → clarifying questions → curated answers → plan document.
2. **Verify (Immediate gate):** Implementation model writes code, runs `task check` (`fmt` → `lint` → `vuln` → `test with -race`).
3. **Accept (Decision gate):** Green → record decisions in `PROGRESS.md` + commit. Red → revert and re-prompt.

This cycle proved more effective than traditional debugging: reverting and re-prompting was faster than diagnosing and fixing generated code, and it kept commit history clean and bisectable.

2 Phase 0 - Ideation, Language Choice, and the Go Pivot

Phase 0 was not a single prompt but a multi-day exploration. Rather than start coding and discover architectural mistakes later, the goal was to make the first few decisions correctly and have the AI design a solid foundation upfront, like language choice, toolchain, and shared testing infrastructure, before any interpreter logic was written.

2.1 The Language Question

The initial question was: what language should be learned by building this project? The goal was to learn something genuinely new. To kick off the exploration, the following prompt was given:

I want to implement an interpreter for Shakespeare Programming Language as a learning project. Suggest some languages that would be well-suited for this task and have strong AI-assisted development support. For each, list the peculiarities, advantages and libraries I would need (CLI, testing, parser generators, documentation).

The shortlist that emerged was Elixir (functional/actor model), Lua (minimal/embeddable), and Ruby (OOP/ecosystem). Ruby was chosen first for its OOP clarity, gem ecosystem (Thor for CLI, Parslet for parser combinators, RSpec for testing, Rake for build automation, YARD for documentation), and alignment with clean, readable code.

2.2 The Go Pivot

Mid-ideation, the long-term viability of Ruby in an AI-driven development world was re-evaluated against Go, so a follow-up prompt asked the model to compare the two languages

from the perspective of a one-person project that would rely heavily on AI code generation.

Compare Ruby and Go for building an SPL interpreter with AI assistance. Consider: static vs. dynamic typing safety for AST manipulation, build and cross-compilation simplicity, standard library completeness for CLI and testing, and the quality of AI-generated Go code vs. Ruby code based on current training data. Also factor in resume relevance and learning curve.

Go won decisively. The key motivations were:

- **Integrated toolchain:** Go ships with a formatter, test runner, dependency manager, and documentation tool - no ecosystem selection fatigue.
- **Static typing:** AST manipulation and expression evaluation are safer with typed structs than Ruby's dynamic dispatch. The compiler catches type errors at build time.
- **Single static binary:** `go build` produces a self-contained executable.
- **Cross-compilation:** `GOOS=windows/linux/darwin GOARCH=amd64` builds for all three platforms from a single command, without `MSYS2` gymnastics.
- **Future-proofing:** Go's ecosystem (Cobra, Docker) makes the project more resume-relevant and maintainable than a Ruby equivalent.

2.3 Scaffolding and Design

Once Go was chosen, the next prompt focused on the shared infrastructure that every phase would depend on:

We are starting an SPL interpreter in Go using a TDD approach with a strict phase-by-phase pipeline (lexer → parser → semantic analysis → runtime). Design the scaffold: a `Taskfile.yaml` with `fmt`, `lint`, `vuln`, `test`, `check` targets; a `.golangci.yaml` with `errcheck`, `govet`, `staticcheck`; a pre-commit hook with `Lefthook`; a `Dockerfile` that produces a static binary; and CI (GitHub Actions) that runs on every push.

This established the entire quality envelope before a single line of interpreter code was written. The pipeline architecture, `lexer → parser → semantic analysis → runtime`, was then formalized with a design document that specified the input and output of each phase.

2.4 Prerequisite Gap Analysis

During scaffolding, the AI flagged two missing blockers that would have derailed any testing effort:

1. The `testdata/` directory for fixture-based testing did not exist.
2. No global `slog` logger was configured.

Both were created before writing any interpreter code. The `testdata/` directory was needed so that every later phase could validate its output against fixed input/expected-output fixtures rather than ad-hoc manual checks; the `slog` logger was needed so that internal state transitions (token emission, parser state, stage mutations) could be inspected during debugging instead of running silently.

Cost: Phase 0 ideation cost \$1.07 in GLM 5.2 queries. There was no separate implementation cost because the output was documentation and infrastructure, not code.

3 Phase 1 and 2 - Lexer and Parser

To keep momentum, these two phases were tackled together in a single plan because their boundaries were well understood and the combined scope still felt manageable. Later phases proved large enough to justify separate plans.

3.1 The Plan-Mode Prompt

The full planning prompt is reproduced below exactly as given to GLM 5.2. It follows the six-section scaffold and enforces the negative constraint that forbids production code:

```
# ROLE & SYSTEM CONTEXT
You are an expert Principal Software Engineer and System Architect
specializing in Compiler Design and the Go (Golang) programming
language. You operate strictly in PLAN MODE. Your job is to create
an incredibly detailed, foolproof, step-by-step execution plan for
implementing a Shakespeare Programming Language (SPL) interpreter
in Go.

The plan must be so granular, explicit, and logical that even a junior
developer or a low-tier LLM could execute it sequentially without
making assumptions or architectural errors.

---

# PROJECT STATUS & ARCHITECTURE
The project setup is already fully completed (module initialized,
folder structure ready, core configuration done). The overall
roadmap is divided into 5 core phases plus future extensions.

Your focus is exclusively on rolling out a comprehensive, granular
plan for **Phase 1 (Lexer)** and **Phase 2 (Parser)**.
```

```
### Available Context Files
You must guide the developer to read, adhere to, and update specific
Markdown files during execution:
1. `progress.md`: Tracks the project state. It must be updated at the
end of every sub-step.
2. `error_taxonomy.md`: Defines the standard error classification and
formatting. All custom errors must follow this.
3. `spl_specification.md`: The official language specification for SPL
syntax, keywords, and behavior.
```

```
### Shared Infrastructure
* **Logger:** A centralized logging system is already in place. Every
component must log its internal state transitions using this logger
.
* **Testing Strategy:** Every single sub-phase must include unit tests
. Use a `testdata/` directory for fixture-based testing (comparing
specific inputs to expected outputs).
```

```
---

# INSTRUCTIONS & PLAN REQUIREMENTS
Generate the execution plan adhering to the following strict rules:
```

```

1. **Unroll Phases 1 & 2:** Break down Phase 1 (Lexer) and Phase 2 (
  Parser) into micro-steps. Do not just say "write the lexer". Break
  it down by tokens, state management, and edge cases.
2. **Test-Driven Approach:** Every micro-step must end with a explicit
  validation/testing phase before moving to the next. Define what
  the `testdata` input and output should look like for that specific
  step.
3. **Context Alignment:** Explicitly state when to consult `
  spl_specification.md`, how to format errors using `error_taxonomy.
  md`, and when to log progress in `progress.md`.
4. **No Omissions:** Do not use placeholders like "// implement the
  rest here". Every keyword, token type, or AST node category for
  these two phases must be accounted for.

---

# EXPECTED OUTPUT FORMAT
Your output must be entirely in English and formatted as a
  hierarchical markdown checklist. For Phase 1 and Phase 2, use the
  following structure:

## Phase X: [Phase Name]
### Objective
[Brief description of what this phase achieves]

### Step X.Y: [Micro-step Name]
* **Goal:** [What exactly needs to be coded]
* **Context Files to Reference:** [e.g., spl_specification.md for
  keyword list]
* **Implementation Details:** [Specific Go types, functions, or
  structures to create. Mention Logger usage]
* **Error Handling:** [How to apply error_taxonomy.md here]
* **Testing & Verification:** [How to test this specific micro-step
  using testdata]
* **Documentation Update:** [Reminder to update progress.md]

```

3.2 Four Clarifying Questions

The plan-mode prompt generated four questions, each resolved through the cross-model triangulation loop:

Question 1.1 - Classification placement. Where should word classification (noun/adjective/character name) live? Decision: **lexer stays dumb, parser classifies**. This kept Phase 1 independently testable: if the lexer needed the noun/adjective dictionary to emit tokens, it could not be tested in isolation until the parser's vocabulary existed, effectively breaking the phase boundary the testing strategy relied on.

Question 1.2 - Dictionary comprehensiveness. How big should the shipped word dictionary be? Decision: a **curated ~80-word dictionary** with fallbacks (unknown noun → value 1, unknown adjective → doubler).

Question 1.3 - Exeunt grammar contradiction. The auto-generated error taxonomy claimed S012 "Exeunt must stand alone", but the canonical Hello World uses [Exeunt Ophelia and Hamlet]. Independent verification against the original SPL manual confirmed both forms

are valid. S012 was corrected **before any code** - a canonical example of the canonical-source adjudication methodology.

Question 1.4 - Case sensitivity. Decision: **case-insensitive keyword matching**, with original lexeme preserved in tokens for debug/output purposes.

3.3 Implementation

DeepSeek V4 Flash followed the plan and implemented both phases.

Plan document: [Phase 1+2 Plan](#).

Costs: Plan \$0.74 (GLM 5.2), Implement \$0.11 (DeepSeek V4 Flash).

4 Phase 3 - Semantic Analysis

4.1 Prerequisite Bugfix Interjection

While preparing the semantic analysis plan, GLM 5.2 independently detected a regression in Phase 2: the `enterSeen` flag was reset at the start of each act, causing false S013 errors on valid cross-act stage persistence (e.g., when characters carried over from Act I without a closing `Exeunt`). The plan explicitly interjected Step 3.0: fix the bug, hoist `enterSeen` to `parseActs()` scope, add a regression fixture (`cross-act-persistence.spl`), run `task check green`, then proceed. This was not an informal aside but became an actual numbered first step inside the Phase 3 plan document itself, so the bugfix was tracked, tested, and committed as its own atomic unit before any semantic-analysis code was written.

4.2 The Plan-Mode Prompt

The full planning prompt is reproduced below exactly as given to GLM 5.2. It follows the six-section scaffold and enforces the negative constraint that forbids production code:

```
# ROLE & SYSTEM CONTEXT
You are an expert Principal Software Engineer and System Architect
specializing in Compiler Design and the Go (Golang) programming
language. You operate strictly in PLAN MODE. Your job is to create
an incredibly detailed, foolproof, step-by-step execution plan for
implementing the **Phase 3 (Semantic Analysis / Type & State
Checker)** for a Shakespeare Programming Language (SPL) interpreter
in Go.

The plan must be so granular, explicit, and logical that even a junior
developer or a low-tier LLM could execute it sequentially without
making assumptions or architectural errors.

---

# PROJECT STATUS & ARCHITECTURE
The project setup, Phase 1 (Lexer), and Phase 2 (Parser) are fully
completed. The Abstract Syntax Tree (AST) has been fully defined
and generated.

Your focus is exclusively on rolling out a comprehensive, granular
plan for **Phase 3 (Semantic Analysis)**. In SPL, semantic analysis
acts as a static/validation pass that ensures the play's narrative
```

```

    logic is sound before execution (e.g., characters cannot speak if
    they are not on stage, declared characters match the official
    dramatis personae, etc.).

### Available Context Files
You must guide the developer to read, adhere to, and update specific
Markdown files during execution:
1. `progress.md`: Tracks the project state. It must be updated at the
   end of every sub-step.
2. `error_taxonomy.md`: Defines the standard error classification and
   formatting. Phase 3 will heavily rely on this for semantic/compile-
   time errors.
3. `spl_specification.md`: The official language specification for SPL
   character limits, operations, and stage rules.

### Shared Infrastructure
* **Logger:** Use the centralized logging system. The Semantic
  Analyzer must log symbol table insertions, lookups, and scope
  entries/exits.
* **Testing Strategy:** Every single sub-phase must include unit tests
  . Use the `testdata/` directory for fixture-based testing (
  comparing valid/invalid AST structures or source files to expected
  semantic error outputs or successful symbol tables).

---

# INSTRUCTIONS & PLAN REQUIREMENTS
Generate the execution plan adhering to the following strict rules:

1. **Unroll Phase 3:** Break down Phase 3 (Semantic Analysis) into
   micro-steps. Do not just say "validate the AST". Break it down into
   :
   * **Symbol Table Construction:** Global declarations (Dramatis
     Personae) and initial values.
   * **Act and Scene Scope Management:** Tracking active characters on
     stage (maximum 2 at any time for dialogues).
   * **Character Validation:** Verifying that a character speaking or
     being spoken to is actually on stage (`Enter`, `Exit`, `Exeunt`
     mechanics).
   * **Expression Verification:** Validating first/second-person
     pronouns resolution based on who is speaking and who is being
     addressed.
   * **Assignment & Operation Checking:** Ensuring comparisons and
     assignments align with SPL rules.
2. **STRICT NEGATIVE CONSTRAINT - NO PRODUCTION CODE:** Do **NOT**
   write full function implementations, logic bodies, or production Go
   code. Your job is to *plan*, not to implement. You may define Go
   struct definitions, interface signatures, or bulleted algorithmic
   logic, but you must never provide complete code blocks meant for
   copy-pasting.
3. **Logic Precision:** While you must not write the code, you must be
   extremely precise about the *logic*. Explain exactly how data
   flows, how the state changes (e.g., how the `Stage` state tracking
   map updates when an `Enter` node is encountered), and what semantic
   rules must be asserted.

```

```

4. **Test-Driven Approach:** Every micro-step must end with an
   explicit validation/testing phase before moving to the next. Define
   what the `testdata` input (invalid/valid SPL code or ASTs) and
   expected semantic error/success should look like.
5. **Context Alignment:** Explicitly state when to consult `
   spl_specification.md`, how to format semantic errors using `
   error_taxonomy.md`, and when to log progress in `progress.md`.

---

# EXPECTED OUTPUT FORMAT
Your output must be entirely in English and formatted as a
  hierarchical markdown checklist using the following structure:

## Phase 3: Semantic Analysis
### Objective
[Brief description of what this phase achieves]

### Step 3.Y: [Micro-step Name]
* **Goal:** [What exactly needs to be checked/validated]
* **Context Files to Reference:** [e.g., spl_specification.md for
  stage rules]
* **Implementation Details:** [Specify Go types, AST Visitor
  interfaces, or Symbol Table struct schemas. Describe the exact
  algorithmic logic, lookup rules, and state mutation steps in plain
  English or pseudo-steps. DO NOT WRITE IMPL CODE.]
* **Error Handling:** [How to apply error_taxonomy.md for specific
  semantic failures like "CharacterNotOnStageError"]
* **Testing & Verification:** [How to test this specific micro-step
  using testdata fixtures]
* **Documentation Update:** [Reminder to update progress.md]

```

4.3 Critical Decisions

Using the same cross-model loop, three questions were resolved:

Question 2.1 - Off-stage value reads. Can a character's value be read while off stage? Decision: **CharRefExpr reads are always allowed off-stage** (only the speaker must be on stage). A stricter rule, requiring every referenced character to be currently on stage, would have rejected the canonical Hello World program itself, which reads Romeo's value in Act I Scene II after he has already exited; the canonical reference was treated as authoritative over the stricter reading.

Question 2.2 - Stage clears on act? The spec's auto-generated draft said yes, but `primes.spl`, a second canonical SPL reference program from the original 2001 manual's appendix, alongside Hello World and the Truth Machine, proved otherwise: Act I ends with two characters still on stage and no closing Exeunt, and Act II Scene I opens with no `[Enter]` at all, meaning those characters could only speak if they had carried over from the previous act. So the decision is that **stage persists across act boundaries**. This reversed the original parser assumption, which had been resetting the stage state at every act.

Question 2.3 - Error reporting mode. Decision: **collect-all (linter-style)**. M-codes are the semantic analyzer's numbered error identifiers (M001 through M008), defined in the error taxonomy. Collect-all means the analyzer does not stop at the first error it finds but it keeps

checking the rest of the program and returns every semantic error from a single run, so a user sees all problems at once instead of fixing one and re-running repeatedly to find the next.

4.4 Implementation

DeepSeek V4 Flash implemented Phase 3.

Plan document: [Phase 3 Plan](#).

Costs: Plan \$1.41 (GLM 5.2), Implement \$0.14 (DeepSeek V4 Flash).

5 Phase 4 - Runtime

Notably, **no clarifying questions were asked** - the specification was complete and unambiguous after Phase 3. When the lexer/parser/semantic decisions are settled, the runtime plan writes itself.

5.1 The Plan-Mode Prompt

The full planning prompt is reproduced below exactly as given to GLM 5.2. It follows the six-section scaffold and enforces the negative constraint that forbids production code:

ROLE & SYSTEM CONTEXT

You are an expert Principal Software Engineer and System Architect specializing in Compiler Design, Virtual Machines, and the Go (Golang) programming language. You operate strictly in PLAN MODE. Your job is to create an incredibly detailed, foolproof, step-by-step execution plan for implementing Phase 4 (Runtime / Evaluator) for a Shakespeare Programming Language (SPL) interpreter in Go.

You must write this execution plan as a new markdown document designed to be saved directly under:

`docs/superpowers/plans/2026-07-12-phase4-runtime.md`

The plan must be so granular, explicit, and logical that even a junior developer or a low-tier LLM could execute it sequentially without making assumptions or architectural errors.

PROJECT STATUS & ARCHITECTURE

The project setup, Phase 1 (Lexer), Phase 2 (Parser), and Phase 3 (Semantic Analysis) are fully completed. We have a fully validated AST, populated symbol tables, and validated jump targets (Acts/Scenes registries).

Your focus is exclusively on rolling out a comprehensive, granular plan for Phase 4 (Runtime/Evaluator).

Available Context Files to Reference during Plan Execution

- progress.md: Tracks the project state. It must be updated at the end of every sub-step.
- error_taxonomy.md: Defines standard error classification. You will define runtime panic/error formats here if needed.
- spl_specification.md: The official language specification for character state, stacks, math operations, and I/O.
- docs/superpowers/plans/2026-07-09-lexer-parser.md (Prior context)
- docs/superpowers/plans/2026-07-10-phase3-semantic-analysis.md (Prior context)

INSTRUCTIONS & PLAN REQUIREMENTS

Generate the execution plan adhering to the following strict rules:

1. Unroll Phase 4: Break down Phase 4 (Runtime) into micro-steps.
Break it down into:
 - Runtime Environment State (Character variable store: name -> int, Stack storage for Remember/Recall: name -> []int, Stage tracking, and execution pointers/indices for Acts and Scenes).
 - Expression Evaluation Engine (Constant word-value calculation, Unary/Binary math operations, Similes, Pronoun resolution based on current Speaker/Listener context).
 - Statement Execution (Dialogue processing, Assignment statements, Stack operations, I/O handling for Speak/Listen as integers or characters via the abstracted reader/writer).
 - Control Flow & Trampoline (Handling GOTO and conditional 'If so' statement logic via Program Counter / index manipulation using the registries built during semantic analysis).
2. STRICT NEGATIVE CONSTRAINT - NO PRODUCTION CODE: Do NOT write full function implementations, logic bodies, or production Go code. Your job is to plan, not to implement. You may define Go struct definitions, interface signatures, or bulleted algorithmic logic, but you must never provide complete code blocks meant for copy-pasting.
3. Logic Precision: Explain exactly how data flows, how the evaluation stack or character maps change, and how execution control transitions between scenes.
4. Test-Driven Approach: Every micro-step must end with an explicit validation/testing phase. Define what the testdata input and expected output/state should look like.
5. Context Alignment: Explicitly state when to consult specification files and when to log progress in progress.md.

EXPECTED OUTPUT FORMAT

Your output must be the raw Markdown content of the file `docs/superpowers/plans/2026-07-12-phase4-runtime.md`. Start directly with the document title. Use the following structure inside the document:

```
# Plan: Phase 4 - Runtime / Evaluator (2026-07-12)
```

```
## Objective
```

```
[Brief description of what this phase achieves]
```

```
## Step 4.Y: [Micro-step Name]
```

- **Goal:** [What exactly needs to be coded/implemented]
- **Context Files to Reference:** [e.g., spl_specification.md]
- **Implementation Details:** [Specify Go types, Environment struct schemas, evaluation loop logic, I/O abstractions. Describe the exact algorithmic steps in plain English or pseudo-steps. DO NOT WRITE IMPL CODE.]
- **Error Handling:** [Runtime panic handling or clean error returns]
- **Testing & Verification:** [How to test this specific micro-step using unit tests or testdata fixtures]
- **Documentation Update:** [Reminder to update progress.md]

5.2 Implementation

DeepSeek V4 Flash implemented Phase 4.

Plan document: [Phase 4 Plan](#).

Costs: Plan \$0.43 (GLM 5.2), Implement \$0.15 (DeepSeek V4 Flash).

6 Phase 5 - CLI Integration

The Phase 5 plan was written directly by DeepSeek V4 Flash (\$0.20) because this phase was more about wiring existing components than pure architecture. What seemed like a simple CLI addition turned out to be more involved, especially the REPL: I wanted a fully interactive REPL but was determined not to modify the interpreter's clean pipeline just to accommodate it.

6.1 The REPL Interaction Model Question

The plan forced a decision among three interaction models:

- **line-at-a-time** (each typed line is parsed and evaluated as a standalone fragment, with character state persisting between lines)
- **full program buffer** (the user pastes or types an entire Act/Scene at once, terminated by an explicit marker, and the whole thing is parsed and run together)
- **hybrid multi-line** (a lightweight prompt convention where special commands like `.act/` `.scene` switch context while individual lines are still evaluated one at a time)

All three assumed a way to inject or persist state between submissions, but the runtime's `env` was unexported with no such API, and the parser only exposed a full-program `Parse()` with no fragment-level entry point, so none of the three could be implemented without modifying already-stable internal packages. The solution that emerged instead was the **Replay-based Accumulating Buffer Model** where every submit re-runs the entire pipeline (`lex` → `parse` → `analyze` → `execute`) on the growing buffer, reusing the exact same tested code path as `spl run` instead of requiring any new state-injection surface.

Deterministic stdin replay kept `Listen/Open mind` calls non-interactive on re-runs: since every submission re-executes the whole program from the start, a naive replay would re-prompt the user for input they had already provided in an earlier turn. Instead, the REPL records every value the user typed in response to a prior `Listen/Open mind` call and automatically feeds that same recorded sequence back into the program on each replay, only pausing to ask the user when genuinely new input is needed beyond what was already recorded.

6.2 Six Subcommands

Command	Purpose
<code>spl run <file></code>	Full pipeline execution
<code>spl tokens <file></code>	Lex only, dump token stream
<code>spl ast <file></code>	Parse only, dump AST as JSON
<code>spl repl</code>	Interactive REPL
<code>spl version</code>	Plain-text version/commit/date
<code>spl about</code>	ASCII art, language credits and authorship

Two global flags apply across all subcommands: `-debug` raises the logger to debug level, printing internal state transitions (token emission, parser state, stage mutations); `-trace` implies

`-debug` and additionally prints four pipeline-stage markers (`- TOKENS -`, `- AST -`, `- SEMANTIC -`, `- EXECUTE -`) to `stderr`, making it possible to see exactly what each phase produced during a single run.

6.3 Implementation

DeepSeek V4 Flash implemented Phase 5.

Plan document: [Phase 5 Plan](#).

Costs: Plan \$0.20, Implement \$0.27 (both DeepSeek V4 Flash).

7 Phase 6 - Documentation

Planned and implemented directly by DeepSeek V4 Flash with no clarifying questions.

An MkDocs Material slate site was deployed to GitHub Pages at <https://lorenzobandini.github.io/shakespeare-interpreter-go/>.

Plan document: [Docs Plan](#).

8 Phase 7 - The REPL-Resolution Crisis

After the initial REPL implementation, testing with real programs revealed sporadic S002, S006, S007, S009, L002, and M004 errors on multi-line pastes and incremental entry. I first questioned whether a REPL was even viable for a language as verbose and stateful as SPL, so the Phase 7 plan opened with an explicit “honest viability verdict”:

Explicitly declare if a viable, clean architectural solution is impossible due to SPL’s grammatical constraints. In that case, halt. We will deprecate and remove the REPL entirely.

The model confirmed the failures were localised buffer/skeleton-splicing bugs, not a fundamental incompatibility. I then collected several real REPL error logs and fed them to GLM 5.2 in a dedicated troubleshooting prompt.

8.1 The Fixes

Six corrective measures were planned and implemented:

1. **Phase-tracking state machine** - A `replPhase` enum tracks whether the input is still in the title, character declarations, body, or finished. A `classifyLine` function uses cheap substring/regex, not full parsing.
2. **Lazy, correctly-placed skeleton injection** - The skeleton (Title + Act I + Scene I) is injected only at the first body submission, and spliced after the last character-declaration byte, never before.
3. **Correct auto-declaration splice ordering** - Only names the user actually references are declared, in first-seen order, with case-insensitive dedup.
4. **Robust incomplete-input detection** - Continuation prompts are triggered only by unterminated brackets, trivially short blocks, or an open-expression heuristic (capped at 40 lines).
5. **Rollback correctness** - A counting writer tracks byte-length output deltas; on failure, buffer and stdin cursor are restored atomically.

6. **Regression fixtures** - Added under `testdata/repl/` covering all known failure patterns; the standard run path was verified unchanged via golden snapshot.

8.2 Implementation

Plan by GLM 5.2, implementation by DeepSeek V4 Flash.

Plan document: [REPL Resolution Plan](#).

Costs: Plan \$0.17 (GLM 5.2), Implement \$0.13 (DeepSeek V4 Flash).

9 Phase 8 - Final Comprehensive Audit

To confirm everything was functioning, spec-compliant, and free of leftover issues, the entire codebase was submitted to a final plan-mode audit by the frontier model. The audit found 2 critical bugs, 6 medium issues, and 6 low-priority hygiene items.

9.1 Implementation

Plan by GLM 5.2, implementation by DeepSeek V4 Flash.

Plan document: [Audit Plan](#).

Costs: Plan \$0.99 (GLM 5.2), Implement \$0.15 (DeepSeek V4 Flash).

10 Cost Ledger Summary

The planner/executor cost optimization strategy is validated by the numbers: planning (GLM 5.2) cost approximately \$5.01, while implementation (DeepSeek V4 Flash) cost approximately \$0.95.

Phase	Plan Model	Plan Cost	Impl. Model	Impl. Cost
Phase 0 - Ideation	GLM 5.2	\$1.07	-	-
Phase 1+2 - Lexer/Parser	GLM 5.2	\$0.74	DS V4 Flash	\$0.11
Phase 3 - Semantic Analysis	GLM 5.2	\$1.41	DS V4 Flash	\$0.14
Phase 4 - Runtime	GLM 5.2	\$0.43	DS V4 Flash	\$0.15
Phase 5 - CLI Integration	DS V4 Flash	\$0.20	DS V4 Flash	\$0.27
Phase 7 - REPL Resolution	GLM 5.2	\$0.17	DS V4 Flash	\$0.13
Phase 8 - Final Audit	GLM 5.2	\$0.99	DS V4 Flash	\$0.15
Total		\$5.01		\$0.95